



INDIAN INSTITUTE OF TECHNOLOGY DELHI

Department of Electrical Engineering
Hauz Khas, New Delhi-110016.

Thursday 10th September, 2026

VLSI DESIGN LABORATORY I – ASSIGNMENT 1

Design and Physical Implementation of a 4×4 Systolic Matrix Multiplication Engine

Group Members:

Rupak Anish Antani

Ankit

2026JVL2631

2026EEN2405

Contents

0.1	Week 1 Updates:	v
0.2	Week 2 Updates:	v
0.3	Week 3 Updates:	v
0.4	Week 4 Updates:	v
0.5	Week 5 Updates:	v
0.6	Week 6 Updates:	vi
1	Introduction	1
1.0.1	Problem Statement	2
1.0.2	Design Specifications	3
1.0.3	Functional Requirements	3
1.0.4	Systolic Matrix Multiplication Engine	4
1.0.4.1	Cycle Accurate Operation	5
1.0.4.2	Processing Element	6
1.0.4.3	Adder	6
1.0.4.4	Multiplier	6
1.0.4.5	Registers	7
1.0.4.6	Top-Level Signals	7
1.0.4.7	Reset Behaviour	7
1.0.4.8	Start Sequence	8
1.0.4.9	Input Skewing	8
1.0.5	Project Timeline	9
2	Literature Review, Design Modelling and Planning	10
2.1	Literature Survey	11
2.1.1	Systolic Array	11
2.1.2	Processing Element (PE)	12
2.1.3	Adder	12
2.1.4	Multiplier	13
2.1.5	Input Skewing	13
2.1.6	Recent Research	13
2.2	Architecture Design	14
2.2.1	Han Carlson Adder	14
2.2.2	Radix-4 Booth Multiplier	15
2.2.3	On-Chip Input Skewing	17

2.2.4	Counter-based Control Signals Block	18
2.3	Logic Modelling and Simulation	18
2.4	Floorplanning	18
2.4.1	Processing Element (PE)	19
2.4.2	Systolic Matrix Multiplication Engine	19
2.5	Metal Planning	20
2.6	Chip Area Estimation	20
3	Schematic Design and Verification	21
3.1	Introduction	22
3.2	Basic CMOS Logic Cells	22
3.2.1	Basic Cell Schematics	22
3.2.2	Performance of Basic Logic Cells	23
3.3	D Flip-Flop	23
3.4	Han–Carlson Adder	24
3.5	Radix-4 Booth Multiplier	24
3.6	Processing Element	25
3.7	Input Skew Block	25
3.8	Control Block	25
3.9	4 × 4 Systolic Array	26
3.9.1	Functional Verification	26
3.9.1.1	Han–Carlson Adder Verification	26
3.9.1.2	Radix-4 Booth Multiplier Verification	27
3.9.1.3	Complete 4 × 4 Systolic Array Verification	27
3.9.1.3.1	Test Case 1: Zero Input	28
3.9.1.3.2	Test Case 2: Identity Matrix	28
3.9.1.3.3	Test Case 3: Positive and Negative Operands	28
3.9.1.3.4	Test Case 4: Sparse Matrix	28
3.9.1.3.5	Test Case 5: Maximum and Minimum Signed Values	29
3.10	Schematic-Level Performance Summary	29
4	Layout Design	30
4.1	Basic CMOS Gate Layout	31
4.1.1	Layout Dimensions and Area	32
4.1.2	Design Rule Check	32
4.1.3	Layout Versus Schematic Verification	33
4.2	Han–Carlson Adder Layout	34
4.2.1	Black Cell Layout	35
4.2.2	Gray Cell Layout	35
4.2.3	Han–Carlson Adder Layout	35
4.2.4	Layout Dimensions and Area	36
4.2.5	DRC Verification	36
4.2.6	LVS Verification	36
4.3	Multiplier Layout	37

4.3.1	Half Adder Layout	37
4.3.2	Full Adder Layout	37
4.3.3	Layout Dimensions and Area	38
4.3.4	DRC Verification	38
4.3.5	LVS Verification	38
4.4	Summary of Layout Implementation	39
5	Conclusion	40
5.1	Timing Path Analysis	41
5.2	Future Work	41

0.1. Week 1 Updates:

- Literature Survey
- Design Architecture
- Logic Simulation
- Chip Area Estimation
- Floor Planning
- Note: All simulations, testbenches, and schematics are available at:
 - `/afs/iitd.ac.in/user/j/jv/jv1262631/CC/VLSI_test`
 - `/afs/iitd.ac.in/user/e/ee/een262405/test/AIC_TEST`

0.2. Week 2 Updates:

- Logic Gates Schematic
- D-Flip Flop Schematic
- Adder Schematic
- Multiplier Schematic
- Power Planning

0.3. Week 3 Updates:

- PE Schematic
- Input Skew Block Schematic
- Control Logic Block Schematic
- Systolic Matrix Multiplication Engine Schematic

0.4. Week 4 Updates:

- Testing of Systolic Engine Schematic
- Layout of Logic Gates
- Patent Survey

0.5. Week 5 Updates:

- Layout of Black cell
- Layout of Gray cell
- Layout of Adder
- Layout of Half Adder
- Layout of Full Adder
- Layout of 8-to-1 Mux
- Patent Survey

0.6. Week 6 Updates:

- Layout of Adder
- Layout of Multiplier
- Layout of PE
- Patent Survey
- Report



CHAPTER

I

INTRODUCTION

Chapter I

Introduction

Matrix multiplication is a fundamental operation in mathematics and computer science, underlying applications such as artificial intelligence (AI), machine learning (ML), digital signal processing (DSP), scientific computing, computer graphics, and robotics. Since these applications often require billions of multiply–accumulate (MAC) operations, efficient matrix multiplication is essential for high computational performance.

Although general-purpose processors can perform matrix multiplication, their limited parallelism restricts throughput for large workloads. Dedicated hardware accelerators overcome this limitation by executing multiple arithmetic operations simultaneously, improving speed and energy efficiency.

Systolic arrays are well suited to matrix multiplication because of their regular structure, local data movement, and high throughput. They consist of identical Processing Elements (PEs) that perform multiply–accumulate operations while forwarding data to neighbouring elements.

1.0.1 Problem Statement

The objective is to design a custom 4×4 systolic matrix multiplication engine for signed integer matrices. Given input matrices A and B , the hardware computes the output matrix C as

$$C = AB, \quad (1.1)$$

where

$$A, B, C \in Z^{4 \times 4}, \quad (1.2)$$

and each element of the output matrix is given by

$$c_{ij} = \sum_{k=0}^3 a_{ik} b_{kj}. \quad (1.3)$$

The computation uses sixteen identical MAC processing elements arranged as a 4×4 systolic array. Each PE accumulates the required products for one output element while forwarding operands to neighbouring PEs.

Example:

Consider the matrices

$$A = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 0 & 1 & 2 & 3 \\ 1 & 0 & 1 & 2 \\ 2 & 1 & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 0 & 2 & 1 \\ 2 & 1 & 0 & 1 \\ 1 & 3 & 1 & 0 \\ 0 & 2 & 1 & 2 \end{bmatrix}.$$

The first element of the output matrix is computed as

$$\begin{aligned} c_{00} &= a_{00}b_{00} + a_{01}b_{10} + a_{02}b_{20} + a_{03}b_{30} \\ &= (1)(1) + (2)(2) + (3)(1) + (4)(0) = 8. \end{aligned} \quad (1.4)$$

The remaining fifteen elements are computed in parallel to obtain the complete output matrix C .

1.0.2 Design Specifications

The design specifications for the proposed 4×4 systolic matrix multiplication engine are summarized in Table 1.1.

Table 1.1: Design Specifications

Parameter	Specification
Technology	TSMC 65nm
Supply Voltage	1.2 V
Input Word Length	4-bit signed (Two's Complement)
Product Width	8 bits
Accumulator Width	10 bits
Matrix Dimensions	4×4
Number of Processing Elements	16
Clocking	Single-phase synchronous clock
Reset	Active-low synchronous reset
Arithmetic	Signed Integer Arithmetic
Target Clock Frequency	100 MHz (Post-PEX)

The 4-bit signed two's-complement operands range from -8 to 7 . Products therefore range from -56 to 64 . Since an 8-bit signed value spans -128 to 127 , an 8-bit product register is sufficient. Each PE also computes the sum of four products,

$$c_{ij} = \sum_{k=0}^3 a_{ik}b_{kj}. \quad (1.5)$$

The four-product accumulation ranges from -224 to 256 . A signed 9-bit value ends at 255 , so it cannot represent 256 . A signed 10-bit value ranges from -512 to 511 ; hence, a 10-bit accumulator is selected to prevent overflow.

1.0.3 Functional Requirements

The proposed engine shall satisfy the following functional requirements:

- Correctly accept the prescribed skewed input streams for matrices A and B at the array

boundaries.

- Perform signed matrix multiplication on two 4×4 input matrices and compute all sixteen output elements accurately.
- Correctly process positive, negative, and zero-valued operands using 4-bit two's-complement arithmetic.
- Execute one signed multiply–accumulate (MAC) operation per valid clock cycle in each processing element.
- Propagate matrix A operands horizontally and matrix B operands vertically through the systolic array while maintaining the required timing relationship.
- Clear all processing element accumulators and internal control registers before the start of every new matrix multiplication operation.
- Operate synchronously using a single-phase clock with an active-high synchronous reset.
- Generate valid busy and done signals to indicate the execution status and successful completion of the computation.
- Prevent acceptance of a new start command while a matrix multiplication is already in progress.
- Retain the computed output matrix until either a new computation is initiated or a reset signal is asserted.
- Produce functionally correct results during both pre-layout and post-layout (PEX) simulations.
- Meet the target operating frequency of 100 MHz after parasitic extraction while maintaining functional correctness.

1.0.4 Systolic Matrix Multiplication Engine

A systolic matrix multiplication engine exploits parallelism and data reuse through a two-dimensional array of identical *Processing Elements (PEs)*. Each PE performs a multiply–accumulate (MAC) operation and communicates with neighbouring PEs. The term *systolic* refers to the rhythmic movement of data through the array. Unlike processor-based implementations that repeatedly fetch operands, a systolic array streams data through PEs and reuses operands across neighbours, reducing memory accesses and interconnect overhead while improving throughput.

The proposed architecture contains 4×4 identical PEs in a regular grid, as shown in Figure 1. Matrix A enters from the left and propagates horizontally, while matrix B enters from the top and propagates vertically. When corresponding operands meet, their signed product is accumulated locally. After four MAC operations, each PE contains one output element, allowing all sixteen elements to be computed in parallel.

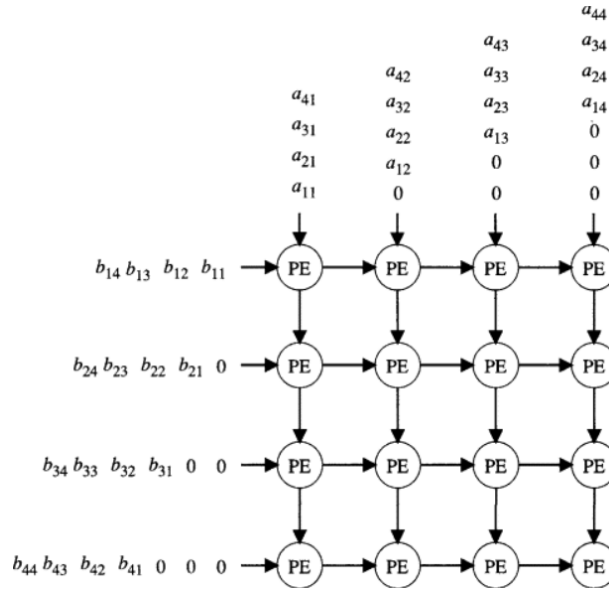


Figure 1.1: Architecture of the proposed 4×4 systolic matrix multiplication engine.

The main advantages of the systolic architecture are:

- High computational throughput due to parallel execution of multiple MAC operations.
- Excellent data reuse, resulting in significantly reduced memory bandwidth requirements.
- Local nearest-neighbour communication minimizes long interconnects and routing complexity.
- High processing element utilization, leading to improved energy efficiency for dense matrix computations.

Its main limitations are:

- Initial latency is introduced due to operand propagation and array filling.
- Larger arrays require increased silicon area and routing resources.
- Input operands must be appropriately skewed or delayed to ensure correct synchronization across the array.

1.0.4.1 Cycle Accurate Operation

The two input matrices considered for the 4×4 matrix multiplication are

$$A = \begin{bmatrix} a_{11} & a_{12} & a_{13} & a_{14} \\ a_{21} & a_{22} & a_{23} & a_{24} \\ a_{31} & a_{32} & a_{33} & a_{34} \\ a_{41} & a_{42} & a_{43} & a_{44} \end{bmatrix} \quad B = \begin{bmatrix} b_{11} & b_{12} & b_{13} & b_{14} \\ b_{21} & b_{22} & b_{23} & b_{24} \\ b_{31} & b_{32} & b_{33} & b_{34} \\ b_{41} & b_{42} & b_{43} & b_{44} \end{bmatrix}$$

Matrix A is streamed from the left and matrix B from the top. Each PE performs one MAC per clock cycle; representative cycle-by-cycle operations are summarized in Table 1.2.

Table 1.2: Cycle-accurate operation of representative Processing Elements (PEs)

Clock	PE(1,1)	PE(1,2)	PE(2,1)	Valid Output
t_0	Reset ($Acc = 0$)	Reset	Reset	None
t_1	Receives a_{11} and b_{11} . Computes $a_{11}b_{11}$.	No valid data.	No valid data.	None
t_2	Receives a_{12} and b_{21} . Updates $Acc = a_{11}b_{11} + a_{12}b_{21}$. Forwards a_{12} and b_{21} .	Receives a_{11} and b_{12} . Computes $Acc = a_{11}b_{12}$.	Receives a_{21} and b_{11} . Computes $Acc = a_{21}b_{11}$.	None
t_3	Receives a_{13} and b_{31} . Updates $Acc = Acc + a_{13}b_{31}$.	Receives a_{12} and b_{22} . Updates $Acc = Acc + a_{12}b_{22}$.	Receives a_{22} and b_{21} . Updates $Acc = Acc + a_{22}b_{21}$.	None
t_4	Receives a_{14} and b_{41} . Updates $Acc = Acc + a_{14}b_{41}$.	Continues MAC operation.	Continues MAC operation.	C_{11} becomes valid.
t_5+	Starts next computation or remains idle.	Continues MAC operation.	Continues MAC operation.	Remaining outputs become valid according to the systolic wavefront.

1.0.4.2 Processing Element

The Processing Element (PE) is the fundamental computational block of the engine. Sixteen identical PEs form the 4×4 array, providing a modular architecture in which each PE computes one output element through signed MAC operations.

As shown in 1.2, each PE receives an A operand from the left and a B operand from the top. The operands are registered, multiplied, and added to the local accumulator. They are simultaneously forwarded rightward and downward to maintain continuous data propagation.

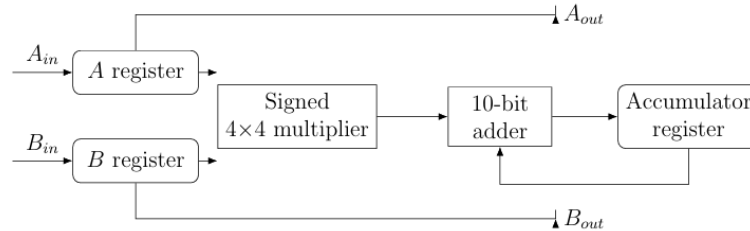


Figure 1.2: Block diagram of the proposed Processing Element (PE).

All PEs operate synchronously, enabling parallel MAC operations with high throughput and low interconnect complexity. Since the PE architecture is identical throughout the array, one verified PE can be replicated sixteen times.

1.0.4.3 Adder

The PE uses a 10-bit accumulator adder. Each cycle, the 8-bit signed product is added to the stored 10-bit partial sum and written back to the accumulator, implementing the MAC operation.

1.0.4.4 Multiplier

The multiplier receives two signed 4-bit two's-complement operands and generates an 8-bit product for the accumulator adder. Its operation directly affects PE throughput, area, power, and timing.

1.0.4.5 Registers

Each PE uses two 4-bit operand registers and one 10-bit accumulator register. The operand registers synchronize incoming data and forwarding, while the accumulator stores the updated partial sum.

1.0.4.6 Top-Level Signals

Table 3 lists the top-level signals of the systolic matrix multiplication engine.

Table 1.3: Top-Level Signals

Signal	Width	Direction	Description
clk	1	Input	System clock.
reset	1	Input	Active-high synchronous reset.
start	1	Input	One-cycle command initiating a new matrix multiplication.
a_row0	4	Input	Left-boundary input to array row 0.
a_row1	4	Input	Left-boundary input to array row 1.
a_row2	4	Input	Left-boundary input to array row 2.
a_row3	4	Input	Left-boundary input to array row 3.
b_col0	4	Input	Top-boundary input to array column 0.
b_col1	4	Input	Top-boundary input to array column 1.
b_col2	4	Input	Top-boundary input to array column 2.
b_col3	4	Input	Top-boundary input to array column 3.
input_valid	1	Input	Indicates that the boundary input values applied during the current clock cycle are valid.
busy	1	Output	Indicates that a matrix multiplication is currently in progress.
done	1	Output	Pulses high for one clock cycle when all sixteen output values become valid.
c00–c33	10 each	Output	Sixteen signed 10-bit matrix multiplication outputs.

1.0.4.7 Reset Behaviour

When reset is asserted, the engine is initialized to its default state:

- All operand registers used for propagating matrix elements through the systolic array are cleared.
- The accumulator register within each of the sixteen Processing Elements (PEs) is reset to zero.
- The internal cycle counter is reinitialized to its starting value.
- The busy and done status signals are deasserted, indicating that no computation is currently in progress and no valid result is available.
- Any output data generated from a previous matrix multiplication is invalidated to prevent incorrect result interpretation.

1.0.4.8 Start Sequence

The operation begins when start is asserted:

- The testbench asserts the start signal for one clock cycle to initiate a new matrix multiplication operation.
- Upon detecting the start signal, all accumulator registers within the sixteen Processing Elements (PEs) are cleared to remove any residual values from previous computations.
- The busy signal is asserted to indicate that the systolic array is actively performing a matrix multiplication.
- The first set of skewed input operands corresponding to matrices A and B is applied to the left and top boundaries of the systolic array.
- The input_valid signal remains asserted throughout the seven input cycles to indicate that the applied boundary data is valid for processing.
- After all partial products have been accumulated and the complete output matrix has been computed, the done signal is asserted for one clock cycle to indicate successful completion of the operation. Simultaneously, the busy signal is deasserted, indicating that the engine is ready for a new computation.

A new start request is ignored while busy is asserted.

1.0.4.9 Input Skewing

The inputs must be skewed so corresponding elements of A and B arrive simultaneously at each PE. Without this synchronization, the required operand pairs would not meet correctly.

For synchronization, the i^{th} row of A is delayed by i cycles and the j^{th} column of B by j cycles. The skewed boundary inputs are:

$$A_{\text{left},i}(t) = \begin{cases} a_{i,t-i}, & 0 \leq t-i < 4, \\ 0, & \text{otherwise,} \end{cases} \quad (1.6)$$

and

$$B_{\text{top},j}(t) = \begin{cases} b_{t-j,j}, & 0 \leq t-j < 4, \\ 0, & \text{otherwise.} \end{cases} \quad (1.7)$$

As shown in Tables 4 and 5, zero operands are applied during inactive cycles. They propagate without affecting accumulation, allowing controlled array fill and drain while maintaining correct data synchronization.

Table 1.4: Matrix- A Boundary Schedule

Row	Input Cycle						
	6	5	4	3	2	1	0
a_row0	0	0	0	a_{03}	a_{02}	a_{01}	a_{00}
a_row1	0	0	a_{13}	a_{12}	a_{11}	a_{10}	0
a_row2	0	a_{23}	a_{22}	a_{21}	a_{20}	0	0
a_row3	a_{33}	a_{32}	a_{31}	a_{30}	0	0	0

Table 1.5: Matrix- B Boundary Schedule

Input Cycle	b_col0	b_col1	b_col2	b_col3
0	b_{00}	0	0	0
1	b_{10}	b_{01}	0	0
2	b_{20}	b_{11}	b_{02}	0
3	b_{30}	b_{21}	b_{12}	b_{03}
4	0	b_{31}	b_{22}	b_{13}
5	0	0	b_{32}	b_{23}
6	0	0	0	b_{33}

1.0.5 Project Timeline

Tasks/Timeline	Week 1	Week 2	Week 3	Week 4	Week 5	Week 6
Literature Survey → 100%						
Literature Survey	100%					
Architecture Design	100%					
Logic Simulation	100%					
Floorplanning + Power planning	100%					
Schematic Design → 100%						
Design + Testing of Gates		100%				
Design + Testing of Registers		100%				
Design + Testing of Adder		100%				
Design + Testing of Multiplier		100%				
Integration + Testing of PE			100%			
Design + Testing of Skew Block			100%			
Design + Testing of Control Block			100%			
Integration + Testing of Systolic Array			100%			
Layout Design and Physical Verification → 56.25 %						
Layout + Checks of Gates				100%		
Layout + Checks of Registers				100%		
Layout + Checks of Adder				100%		
Layout + Checks of Multiplier				100%		
Integration + Checks of PE					50%	
Layout + Checks of Skew Block					0%	
Layout + Checks of Control Block					0%	
Integration + Checks of Systolic Array					0%	
Analysis, Optimization and Conclusion → 50 %						
Parasitic Extraction						0%
Timing Path Analysis						100%
Parasitic Analysis						0%
Report making and Conclusion						100%

Table 1.6: Project Timeline

	Both (Common Tasks)
	Rupak Antani
	Ankit



CHAPTER

II

LITERATURE REVIEW, DESIGN MODELLING AND PLANNING

Chapter II

Literature Review, Design Modelling and Planning

2.1. Literature Survey

2.1.1 Systolic Array

The concept of the systolic array was introduced by *H. T. Kung* and *Charles E. Leiserson* in 1979 as a VLSI architecture that arranges identical Processing Elements (PEs) in a regular grid, allowing data to flow only between neighboring processors. It helps to minimize memory accesses and interconnect complexity while enabling high computational throughput. In 1982, Kung further established the architecture in the landmark paper “*Why Systolic Architectures?*”, highlighting its advantages of local communication, pipelined computation, scalability, and regular VLSI implementation. During the 1980s, systolic arrays were successfully applied to matrix multiplication, convolution, FFT and signal processing.

The popularity of systolic arrays increased significantly with the rise of artificial intelligence. In 2017, Google introduced the Tensor Processing Unit (TPU), whose matrix multiplication engine is based on a 2D systolic array. Today, systolic arrays form the computational core of many modern AI accelerators, with ongoing research focusing on improved dataflows, higher PE utilization, mixed-precision arithmetic, and efficient support for transformer-based workloads. Table 7 mentions some of the important milestones in the development of systolic array architectures.

Table 2.1: Evolution of Systolic Array Architectures

Year	Milestone
1979	Kung and Leiserson introduced the concept of systolic arrays for VLSI systems.
1980	Algorithms for VLSI processor arrays were proposed for efficient parallel computation.
1982	Kung published “Why Systolic Architectures?”, establishing the theoretical foundation of systolic computing.
1982–1987	Development of systolic processors for convolution, signal processing, databases, and the Warp Computer.
1990s	Research activity declined with the widespread adoption of superscalar processors and GPUs.
2017	Google introduced the Tensor Processing Unit (TPU), reviving systolic arrays for deep learning acceleration.
2020–Present	Systolic arrays are extensively used in AI accelerators for CNNs, Transformers, and Large Language Models (LLMs).

2.1.2 Processing Element (PE)

The concept of the Processing Element (PE) evolved alongside the development of systolic arrays, where each PE acts as a simple computational unit performing arithmetic operations while forwarding data to neighboring PEs. Early systolic architectures proposed by *H. T. Kung* employed PEs containing basic arithmetic and storage elements to achieve high throughput through pipelined execution. Over time, PE architectures evolved to include dedicated Multiply–Accumulate (MAC) units, local registers, control logic, and pipelining to improve performance and energy efficiency. Depending on the target application, modern PEs may also support mixed-precision arithmetic, sparsity-aware computation, and configurable dataflows.

2.1.3 Adder

Table 2.2 compares commonly used adder architectures in terms of area, speed, power consumption, and wiring complexity. The numerical values are representative values compiled from published adder comparisons. Values marked with $\approx$ are estimated values used to provide a consistent comparison where directly reported values were not available.

Table 2.2: Comparison of Common Adder Architectures

Adder	Area	Speed	Power	Wiring
Ripple Carry	2214 μm^2	4.208 ns	0.206 mW	≈ 1.0
Carry Lookahead	3000 μm^2	2.80 ns	0.35 mW	≈ 2.5
Carry Skip	2500 μm^2	3.20 ns	0.28 mW	≈ 1.5
Carry Select	6201 μm^2	2.00 ns	1.106 mW	≈ 4.0
Carry Save	4000 μm^2	1.80 ns	0.65 mW	≈ 3.0
Brent–Kung	16969 μm^2	667 ps	14.4 pJ	≈ 2.0
Kogge–Stone	34209 μm^2	548 ps	29.7 pJ	≈ 5.0
Han–Carlson	23950 μm^2	566 ps	20.1 pJ	≈ 3.5

2.1.4 Multiplier

Table 2.3 compares commonly used multiplier architectures in terms of area, speed, power consumption, and wiring complexity. The numerical values are representative values compiled from published multiplier implementations. Values marked with $\approx$ are estimated values used where directly comparable published values were not available.

Table 2.3: Comparison of Common Multiplier Architectures

Multiplier	Area	Speed	Power	Wiring
Array	1323.82 μm^2	14.93 ns	0.066 mW	≈ 1.0
Booth	2800 μm^2	3.75 ns	0.325 mW	≈ 2.0
Wallace Tree	5300 μm^2	1.56 ns	0.656 mW	≈ 4.5
Dadda	3500 μm^2	1.65 ns	0.500 mW	≈ 3.5
Baugh–Wooley	3000 μm^2	2.50 ns	0.450 mW	≈ 2.5
Braun	4000 μm^2	4.00 ns	0.550 mW	≈ 1.5

2.1.5 Input Skewing

Input skewing has been used in systolic arrays to synchronize the arrival of operands at different Processing Elements (PEs). There are several approaches have been proposed for achieving this synchronization:

- **Temporal Skewing:** For matrix multiplication, successive rows of matrix A and columns of matrix B are typically delayed progressively to create the required wavefront of computation.
- **Register-Based Skewing:** Delay registers or shift registers are placed at the array inputs to generate the required skew internally. This simplifies the external input schedule but introduces additional registers and hardware overhead.
- **Memory-Based Skewing:** Operands are stored in buffers or memories and read according to a predetermined address schedule. The required skew is therefore produced by controlling the order and timing of memory accesses rather than by dedicated delay registers.
- **Dynamic Data Scheduling:** More flexible architectures use control logic to dynamically determine the timing and movement of operands. Such approaches can support different matrix dimensions and dataflows, but require additional control complexity compared with fixed skewing.
- **FSM-based Skewing:** In this approach, an FSM is used for each row and column in order to control when should we give input and 0 and when not and which input should be given in which clock cycle.

2.1.6 Recent Research

Recent research on systolic matrix multiplication engines has focused on improving throughput, energy efficiency, and hardware utilization for AI accelerators. The paper “*Accelerating Attention Mechanism on FPGAs based on Efficient Reconfigurable Systolic Array*” presented a reconfigurable systolic array architecture for Transformer attention mechanisms, improving resource utilization and supporting flexible dataflows for FPGA implementations. In “*VerSA: Versatile Systolic Array Architecture for Sparse and Dense Matrix Multiplications*”, a con-

figurable architecture capable of efficiently handling both dense and sparse matrix multiplication was proposed, resulting in improved Processing Element utilization and reduced energy consumption. The work “*High-throughput Systolic Array-based Accelerator for Hybrid Transformer-CNN Networks*” introduced a configurable systolic array capable of accelerating both convolutional and Transformer operations using a unified matrix multiplication framework. More recently, “*DiP: A Scalable, Energy-Efficient Systolic Array for Matrix Multiplication Acceleration*” proposed a novel Diagonal-Input and Permutated (DiP) dataflow that eliminates synchronization FIFOs found in conventional weight-stationary systolic arrays, thereby improving throughput and energy efficiency.

All the research papers and patents that were referred while working on the assignment are mentioned in the table below as well as at the end of the report.

Table 2.4: Patents and Research Papers Related to Systolic Arrays and Layout Design

No.	Patent / Paper Name	Applicant / Authors	Year	Type
1	Increasing Positive Clock Skew for Systolic Array Critical Path	Amazon	2022	Patent
2	Data-Driven Column-Wise Clock Gating of Systolic Arrays	Meta	2024	Patent
3	Floorplan-Optimized Matrix Extension Architecture for Processors	Alibaba	2024	Patent
4	Technique for Distributing Routing into Superfluous Metal Section of an IC	ARM	2019	Patent
5	Hardware Accelerator for Systolic Matrix Multiplication	Habana Labs	2021	Patent
6	Systolic Array of Arbitrary Physical and Logical Depth	Intel	2024	Patent
7	Systolic Array and Accelerator Including the Same	Samsung	2022	Patent
8	Systems and Methods for Reconfigurable Systolic Arrays	Intel	2020	Patent
9	Using Sparsity Metadata to Reduce Systolic Array Power Consumption	Intel	2021	Patent
10	Checksum-Based Fault Detection and Correction for a Matrix Compute Engine	Intel	2022	Patent
11	Clocking a Systolic Array on Both Edges of a Clock Signal	Qualcomm Incorporated	2026	Patent
12	Cross-Mesh Clock Network Synthesis	W.-K. Cheng et al	2023	Paper

2.2. Architecture Design

The entire design consists of 3 major elements - PE, input skewing block and control signals block. Also, the 2 major components of the PE are adder and multiplier. In this project, we have implemented the Han Carlson adder, Radix-4 Booth multiplier, FSM-based on-chip input skewing and counter-based control signals block. Each of these blocks are described in the sections below.

2.2.1 Han Carlson Adder

The Han–Carlson (HC) adder is a parallel-prefix adder that provides a trade-off between speed, area, and wiring complexity. It consists mainly of a *pre-processing stage*, a prefix network

containing *black and grey cells*, and a *post-processing stage*.

In the pre-processing stage, the generate and propagate signals are obtained as

$$G_i = A_i B_i, \quad P_i = A_i \oplus B_i. \quad (2.1)$$

The prefix network combines these signals to determine the carry signals. A *black cell* has two outputs and combines two prefix pairs as

$$G_{i:j} = G_i + P_i G_j, \quad P_{i:j} = P_i P_j, \quad (2.2)$$

whereas a *grey cell* produces only the generate output

$$G_{i:j} = G_i + P_i G_j. \quad (2.3)$$

Thus, black cells are used when both group generate and group propagate signals are required, while grey cells are used near the carry outputs when only the carry-generating signal is required. The HC prefix network uses these cells in a sparse arrangement to reduce the number of cells and wiring compared with a Kogge–Stone adder.

Finally, the sum bits are generated in the post-processing stage as

$$S_i = P_i \oplus C_i, \quad (2.4)$$

where C_i is the carry entering bit i .

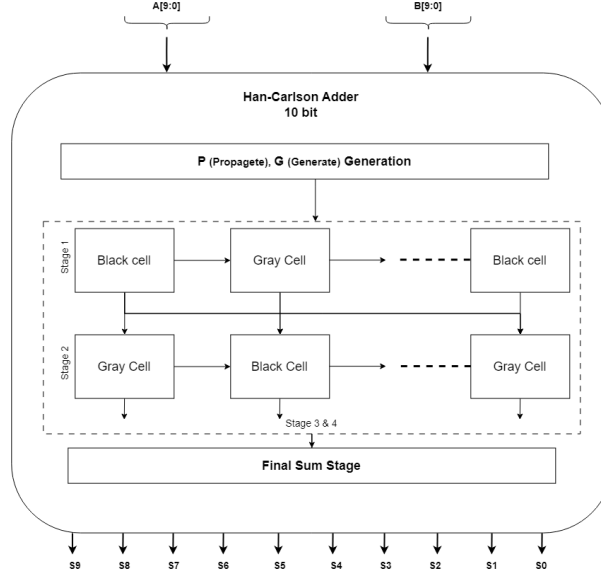


Figure 2.1: Block diagram of Han Carlson Adder

2.2.2 Radix-4 Booth Multiplier

The Radix-4 Booth multiplier is a signed multiplication architecture that reduces the number of partial products by encoding two multiplier bits at a time. It consists mainly of a *Booth encoding stage*, *partial-product generation stage*, and *partial-product addition stage*.

In the Booth encoding stage, the multiplier bits are examined in overlapping groups of three bits $(Y_{2i+1}, Y_{2i}, Y_{2i-1})$, with $Y_{-1} = 0$. Each group is encoded to generate a coefficient from the

set

$$\{-2, -1, 0, +1, +2\}. \quad (2.5)$$

The corresponding partial products are generated as 0 , $\pm X$, or $\pm 2X$, where X is the multiplicand. The partial products are then appropriately sign-extended and shifted by two bit positions for each successive Booth group.

Table 2.5: Booth Encoding Table

Booth Code	Operation	Generated Multiple
000	0	$(0 * A)$
001	+1	$(+A)$
010	+1	$(+A)$
011	+2	$(+2A)$
100	-2	$(-2A)$
101	-1	$(-A)$
110	-1	$(-A)$
111	0	$(0 * A)$

Table 2.6: Booth Operation and Implementation

Operation	Implementation	Description
0	$(0.A = 0)$	No partial product is generated.
+1	$(+A)$	Multiplicand is used directly.
+2	$(A << 1)$	Multiplicand is shifted left by one bit (equivalent to multiplying by 2).
-1	Two's Complement of (A)	Generates the negative of the multiplicand.
-2	Two's Complement of $(A << 1)$	Generates the negative of twice the multiplicand.

The generated partial products are accumulated using a *partial-product reduction tree*, typically implemented using carry-save adders (CSAs), followed by a final carry-propagate adder such as a Han–Carlson or Kogge–Stone adder. Thus, Radix-4 Booth encoding approximately halves the number of partial products compared with conventional binary multiplication, reducing the addition complexity and improving the overall multiplier performance.

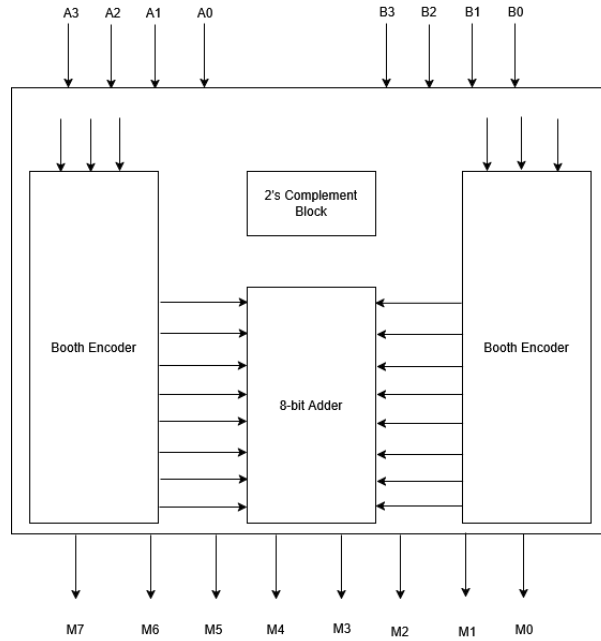


Figure 2.2: Block diagram of Radix-4 Booth Multiplier

2.2.3 On-Chip Input Skewing

An on-chip input skewing block was implemented to provide the required cycle-wise delay to the input data before it is applied to the processing elements. The block uses registers, a counter, and multiplexers to generate the required skew.

The input data is first stored in registers, while the counter determines the required number of delay cycles. Multiplexers select the appropriate registered data according to the programmed skew value. Thus, different processing elements can receive the input operands at the required clock cycles without requiring external cycle-by-cycle input control.

For a skew of k cycles, the input is delayed by k clock cycles before being forwarded to the corresponding processing element. This allows the required temporal alignment of operands for the systolic array operation.

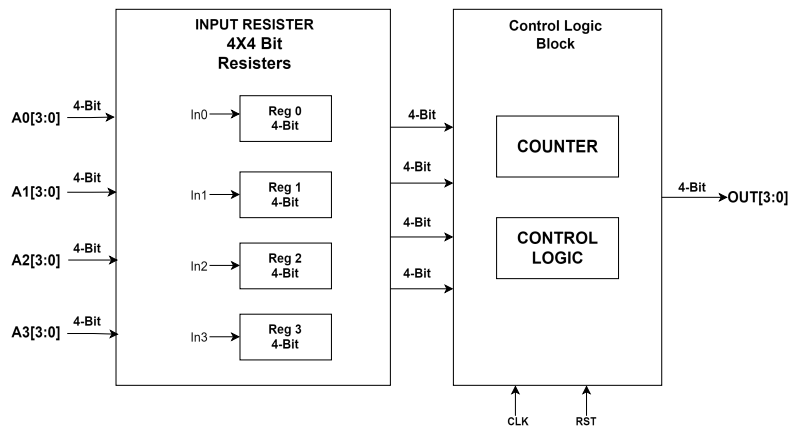


Figure 2.3: On-chip input skewing block.

2.2.4 Counter-based Control Signals Block

The Control signal block has inputs 'rst', 'start', 'inp_valid' and 'clk' and it has outputs 'busy' and 'done'. So, when the 'start' signal comes and the 'inp_valid' signal is high, the counter starts counting from 0 and is reseted at 11. So, the 'busy' signal stays high for the first 10 clock cycles and then the 'done' signal is made high for 1 clock cycle in the last. The block diagram of the control signals block is shown in figure 1.7.

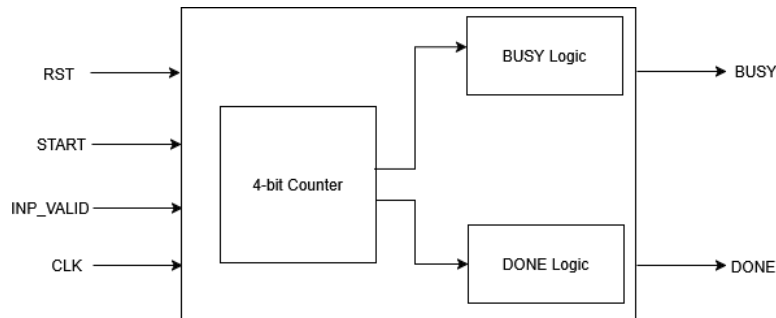


Figure 2.4: Block diagram of Control Signals Block (PE).

2.3. Logic Modelling and Simulation

The entire 4 x 4 Systolic Matrix multiplication engine for implemented in Verilog in a behavioural manner in order to verify the logic of the architecture. The entire design was made in a hierarchical manner and it was tested using several test cases. The output waveform in shown in Figure 1.8

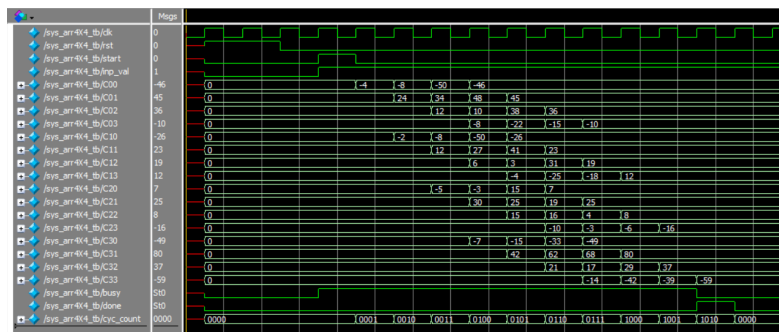


Figure 2.5: Logic Simulation in Verilog

2.4. Floorplanning

2.4.1 Processing Element (PE)

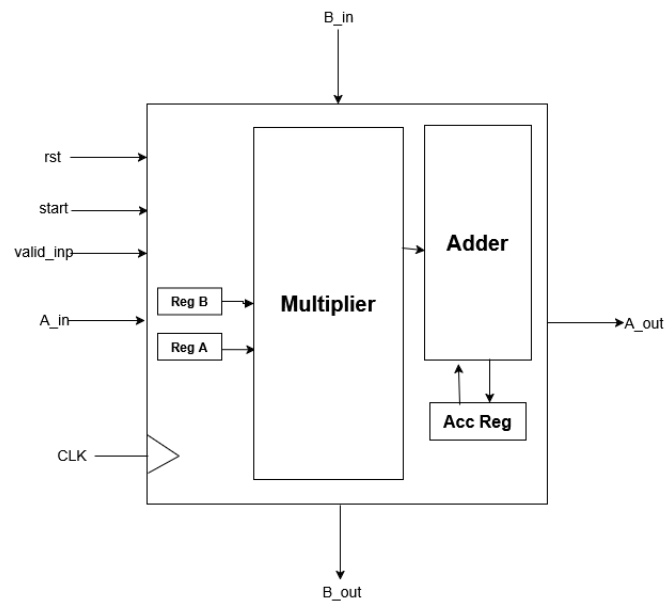


Figure 2.6: Floorplan of the (PE)

2.4.2 Systolic Matrix Multiplication Engine

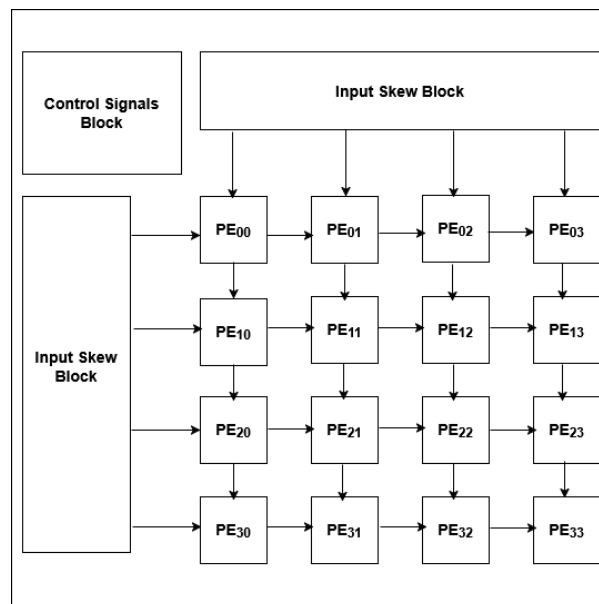


Figure 2.7: Floorplan of the Systolic Matrix Multiplication Engine

2.5. Metal Planning

Table 2.7: Metal Layer Assignment for Different Signals

Metal Layer	Signals	Reason	Orientation
8	VDD	Top metal is wide, so it carries high power with low resistance.	Horizontal
7	GND	Wide metal provides a strong, low-resistance ground path.	Vertical
6	CLK	Higher metal reduces RC delay and clock routing congestion.	Horizontal
5	start, rst, inp_val	Used for important control signals with moderate routing needs.	Vertical
4	Inter-PE connections	Higher layer supports longer connections between PEs.	Horizontal
3	Intra-PE connections	Suitable for shorter connections within a PE.	Vertical
2	Internal routing	Used for local signal connections.	Horizontal
1	Internal routing	Used for short, low-level local routing.	Vertical

2.6. Chip Area Estimation

Table 2.8: Gate Count of Major Components

Component	Number of Gates
D Flip Flop	6
Han Carlson Adder	74
Booth Multiplier	200
Registers in 1 PE	108
1 PE	388
16 PE	6208
Input Skew Block	784
Control Block	40
Systolic Matrix Multiplication Engine	7032

- Assuming 4 transistors/gate: Total number of transistors = $7032 \times 4 = 28,128$
- Assuming $0.40 \mu\text{m}^2$ area/transistor in TSMC 65 nm technology: Total area of transistors = $28,128 \times 0.40 = 11,251 \mu\text{m}^2$
- Assuming chip area = $20 \times$ total transistor area: Total chip area = $20 \times 11,251 = 225,020 \mu\text{m}^2$

$$\text{Total chip area} \approx 0.225 \text{ mm}^2 \quad (2.6)$$

For a square chip:

$$\text{Chip dimensions} = \sqrt{0.225} \approx 0.475 \text{ mm} \times 0.475 \text{ mm} \quad (2.7)$$



CHAPTER

III

SCHEMATIC DESIGN AND
VERIFICATION

Chapter III

Schematic Design and Verification

3.1. Introduction

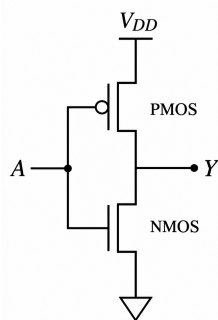
This chapter presents the transistor-level schematic design and schematic-level verification of the major building blocks of the proposed 4×4 systolic matrix multiplication engine. The circuits were designed and simulated in Cadence Virtuoso using the selected CMOS technology.

3.2. Basic CMOS Logic Cells

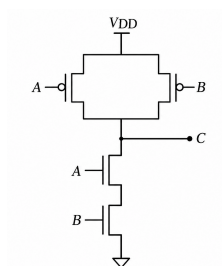
The basic CMOS logic cells used in the design are the NOT, NAND, NOR, and XOR gates. The gates were implemented using `nch_1vt` and `pch_1vt` transistors with a supply voltage of 1.2 V. The initial delay measurements were performed with a 1 fF output load.

3.2.1 Basic Cell Schematics

The transistor-level schematics of the four basic CMOS cells are shown in Figure 3.2.



(a) NOT gate



(b) NAND gate

Figure 3.1: Transistor-level schematics of the NOT and NAND logic cells



Figure 3.2: Transistor-level schematics of the NOR and XOR logic cells

3.2.2 Performance of Basic Logic Cells

The measured timing performance of the basic CMOS cells is given in Table 3.1.

Table 3.1: Performance of basic CMOS logic cells.

Cell	Rise Time	Fall Time	Propagation Delay
NOT	20 ps	21 ps	24 ps
NAND	28 ps	29 ps	32 ps
NOR	26 ps	28 ps	36 ps
XOR	42 ps	40 ps	56 ps

3.3. D Flip-Flop

The D flip-flop is used as a sequential storage element for operand propagation and intermediate data storage in the processing element.

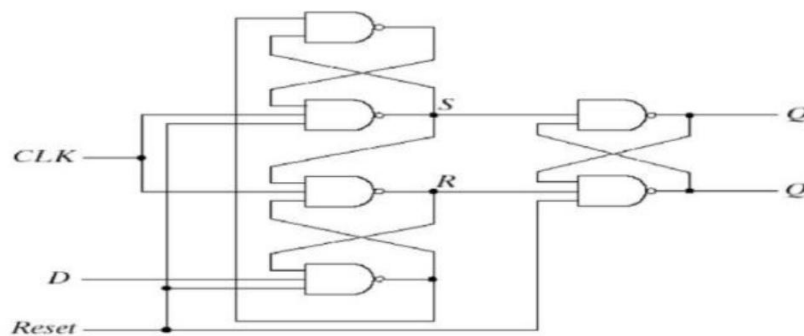


Figure 3.3: D flip-flop schematic.

The measured timing parameters of the D flip-flop are listed in Table 3.2.

Table 3.2: Performance of the D flip-flop.

Parameter	Measured Value
Rise Time	40 ps
Fall Time	36 ps
Setup Time	45 ps
Hold Time	10 ps
Clock-to-Q Delay	46 ps

The D flip-flop was functionally verified by applying the required input and clock transitions and observing the corresponding output response.

3.4. Han–Carlson Adder

The Han–Carlson adder is used as the accumulator adder in the processing element. The block diagram of the Han Carlson adder that is implemented in the schematic is shown in figure 2.1. The complete transistor-level schematic was implemented and simulated in Cadence Virtuoso.

The measured timing performance of the Han–Carlson adder is given in Table 3.3.

Table 3.3: Performance of the Han–Carlson adder.

Parameter	Measured Value
Rise Time	49 ps
Fall Time	48 ps
Propagation Delay	390 ps

The Han–Carlson adder was tested for the required arithmetic corner cases, including zero, positive, negative, mixed-sign, and maximum-magnitude input combinations. The simulated outputs were verified against the expected addition results.

3.5. Radix-4 Booth Multiplier

A signed Radix-4 Booth multiplier was implemented for the multiplication operation in the processing element. The block diagram of the multiplier which was implemented in the schematic is shown in Figure 2.2. The complete transistor-level schematic was implemented and simulated in Cadence Virtuoso.

The measured timing performance of the Radix-4 Booth multiplier is given in Table 3.4.

Table 3.4: Performance of the Radix-4 Booth multiplier.

Parameter	Measured Value
Rise Time	50 ps
Fall Time	53 ps
Propagation Delay	812 ps

The Radix-4 Booth multiplier was tested for the required signed arithmetic corner cases,

including zero, positive, negative, mixed-sign, and maximum-magnitude input combinations. The simulated outputs were verified against the expected multiplication results.

3.6. Processing Element

The processing element (PE) is the basic computational unit of the systolic array. It receives operands from the neighboring PEs, performs multiplication and accumulation, and forwards the operands to the next processing elements. The complete PE schematic is shown in Figure 2.6.

The PE integrates the signed multiplier, accumulator adder, accumulator register, operand registers, and forwarding paths. The measured timing performance of the complete PE is given in Table 3.5.

Table 3.5: Performance of the complete processing element.

Parameter	Measured Value
Rise Time	105 ps
Fall Time	110 ps
Propagation Delay	1.24 ns
Contamination Delay	230 ps

The complete PE was tested using the required arithmetic corner cases. The verification included zero, positive, negative, mixed-sign, and maximum-magnitude operands. The multiply-accumulate results were verified against the expected values.

3.7. Input Skew Block

The input skew block provides the required temporal alignment of the matrix operands before they enter the systolic array. The complete input skew block block diagram that has been implemented in the schematic is shown in Figure 2.3. The required skew delays row i of matrix A by i cycles and column j of matrix B by j cycles. The input skew functionality was verified by observing the cycle-by-cycle delay of the input operands according to the required systolic input schedule.

3.8. Control Block

The control block generates the control signals required for initialization and operation of the systolic array. The block diagram of the control block implemented in the schematic is shown in Figure 2.4. The control functionality was verified for reset, start, busy, input-valid, and done operations.

3.9. 4×4 Systolic Array

The complete systolic matrix multiplication engine consists of sixteen identical processing elements arranged in a 4×4 array. The block diagram of the complete array that has been implemented in the schematic is shown below.

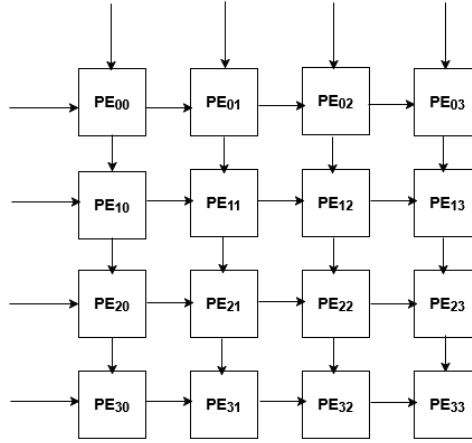


Figure 3.4: Complete 4×4 systolic array schematic.

Each PE receives an operand from matrix A from the left and an operand from matrix B from above. The operands are multiplied and the resulting product is accumulated locally. The A and B operands are then forwarded to the right and downward, respectively.

3.9.1 Functional Verification

The designed arithmetic blocks were functionally verified at the schematic level using representative corner-case input vectors. The verification was performed for the Han–Carlson adder, Radix-4 Booth multiplier, complete processing element (PE), and the complete 4×4 systolic array.

3.9.1.1 Han–Carlson Adder Verification

The Han–Carlson adder is used in the processing element to perform the accumulation of the multiplier output. Since the inputs of the Radix-4 Booth multiplier are 4-bit signed values ranging from -8 to $+7$, the multiplier output ranges from -56 to $+64$.

In the 4×4 systolic array, each processing element can perform up to four multiply-accumulate operations. Therefore, the adder must support the accumulation of multiple multiplier outputs. The maximum positive accumulation is

$$4 \times (-8 \times -8) = 4 \times 64 = 256, \quad (3.1)$$

while the maximum negative accumulation is

$$4 \times (7 \times -8) = 4 \times (-56) = -224. \quad (3.2)$$

The Han–Carlson adder was therefore verified using input combinations corresponding to the multiplier output and the accumulated values encountered in the systolic array. The selected

test cases cover zero, positive, negative, mixed-sign, maximum product, minimum product, and maximum four-term accumulation cases.

Table 3.6: Corner-case input vectors for Han–Carlson adder.

C_{in}	Product	Expected Sum
0	0	0
0	1	1
0	-1	-1
0	64	64
0	-56	-56
64	64	128
-56	-56	-112
64	-56	8
-64	56	-8
192	64	256
-168	-56	-224

The simulated outputs were compared with the expected arithmetic results for all the above cases, including the maximum four-term accumulation limits of the systolic processing element.

3.9.1.2 Radix-4 Booth Multiplier Verification

The Radix-4 Booth multiplier was verified using signed operand combinations covering zero, positive, negative, mixed-sign, and maximum/minimum magnitude cases. The applied input vectors and expected products are listed in Table 3.7.

Table 3.7: Corner-case input vectors for Radix-4 Booth multiplier.

A	B	Expected Product
0	0	0
0	7	0
1	1	1
7	1	7
-1	1	-1
-1	-1	1
7	-1	-7
-8	1	-8
-8	-1	8
7	7	49
-8	-8	64
7	-8	-56
-8	7	-56

The simulated multiplier outputs were compared with the expected signed multiplication results.

3.9.1.3 Complete 4×4 Systolic Array Verification

The complete 4×4 systolic array was functionally verified using matrix multiplication test cases designed to cover the required arithmetic corner cases. The selected test cases include zero inputs, identity operation, positive and negative operands, mixed-sign multiplication, sparse operands, and maximum/minimum signed input values.

For matrices A and B , the expected output was calculated as

$$C = A \times B, \quad (3.3)$$

where each output element is given by

$$c_{ij} = \sum_{k=0}^3 a_{ik} b_{kj}. \quad (3.4)$$

The following test cases were used to verify the complete systolic array.

3.9.1.3.1 Test Case 1: Zero Input This test verifies the zero-input condition and confirms that the multiply-accumulate datapath produces zero when the operands are zero.

$$A = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (3.5)$$

$$C = \begin{bmatrix} 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 0 & 0 & 0 \end{bmatrix} \quad (3.6)$$

3.9.1.3.2 Test Case 2: Identity Matrix This test verifies direct data propagation through the array and positive-value multiplication.

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \quad (3.7)$$

$$C = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \quad (3.8)$$

3.9.1.3.3 Test Case 3: Positive and Negative Operands This test verifies positive multiplication, negative multiplication, and mixed-sign multiplication and accumulation.

$$A = \begin{bmatrix} 1 & -2 & 3 & -4 \\ -1 & 2 & -3 & 4 \\ 5 & -6 & 7 & -8 \\ -5 & 6 & -7 & 8 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & -1 & 2 & -2 \\ 2 & -2 & 3 & -3 \\ 3 & -3 & 4 & -4 \\ 4 & -4 & 5 & -5 \end{bmatrix} \quad (3.9)$$

$$C = \begin{bmatrix} -10 & 10 & -10 & 10 \\ 10 & -10 & 10 & -10 \\ -18 & 18 & -18 & 18 \\ 18 & -18 & 18 & -18 \end{bmatrix} \quad (3.10)$$

3.9.1.3.4 Test Case 4: Sparse Matrix This test verifies multiplication with zero and non-zero operands at different positions in the matrix, thereby testing different PE locations and accumulation paths.

$$A = \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 0 & 2 & 0 \\ 0 & 0 & 0 & 0 \\ 0 & 3 & 0 & 0 \end{bmatrix}, \quad B = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 5 & 6 & 7 & 8 \\ 9 & 10 & 11 & 12 \\ 13 & 14 & 15 & 16 \end{bmatrix} \quad (3.11)$$

$$C = \begin{bmatrix} 1 & 2 & 3 & 4 \\ 18 & 20 & 22 & 24 \\ 0 & 0 & 0 & 0 \\ 15 & 18 & 21 & 24 \end{bmatrix} \quad (3.12)$$

3.9.1.3.5 Test Case 5: Maximum and Minimum Signed Values This test verifies the datapath using the maximum and minimum representable signed input values.

$$A = \begin{bmatrix} 127 & 127 & 127 & 127 \\ -128 & -128 & -128 & -128 \\ 127 & -128 & 127 & -128 \\ -128 & 127 & -128 & 127 \end{bmatrix}, \quad B = \begin{bmatrix} 127 & -128 & 127 & -128 \\ -128 & 127 & -128 & 127 \\ 127 & -128 & 127 & -128 \\ -128 & 127 & -128 & 127 \end{bmatrix} \quad (3.13)$$

$$C = \begin{bmatrix} -128 & -128 & -128 & -128 \\ 128 & 128 & 128 & 128 \\ -16384 & 16384 & -16384 & 16384 \\ 16384 & -16384 & 16384 & -16384 \end{bmatrix} \quad (3.14)$$

The complete 4×4 systolic array was verified using these matrix-level test cases. The selected inputs exercise zero, positive, negative, mixed-sign, sparse, and maximum/minimum magnitude conditions while also testing the multiply-accumulate operation across the PE array.

3.10. Schematic-Level Performance Summary

The measured performance of the major designed blocks is summarized in Table 3.8.

Table 3.8: Summary of schematic-level timing performance.

Block	Rise Time	Fall Time	Propagation Delay
NOT	20 ps	21 ps	26 ps
NAND	28 ps	29 ps	32 ps
NOR	26 ps	28 ps	36 ps
XOR	42 ps	40 ps	56 ps
D Flip-Flop	40 ps	36 ps	120 ps
Han–Carlson Adder	49 ps	48 ps	390 ps
Radix-4 Booth Multiplier	50 ps	53 ps	812 ps
Processing Element	105 ps	110 ps	1246 ps



CHAPTER

IV

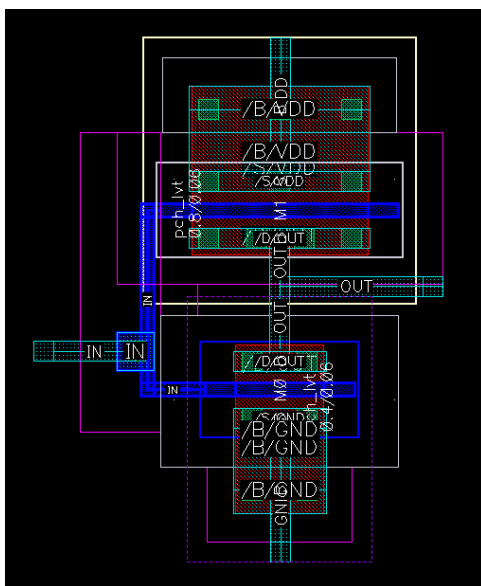
LAYOUT DESIGN

Chapter IV

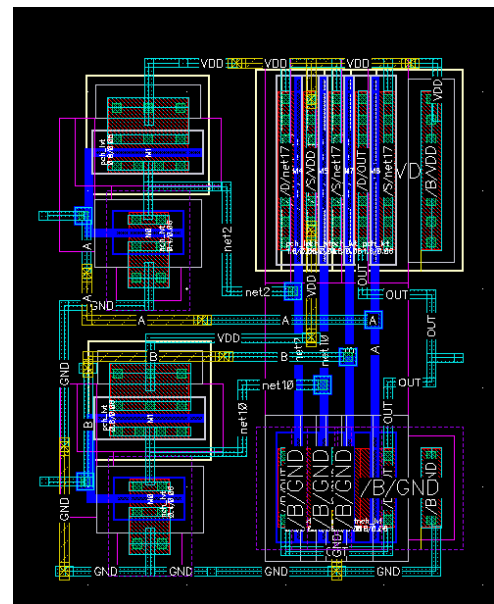
Layout Design

4.1. Basic CMOS Gate Layout

The layouts of the basic CMOS logic cells were implemented in Cadence Virtuoso. The NOT, NAND, NOR, and XOR gates were laid out using the selected technology design rules.

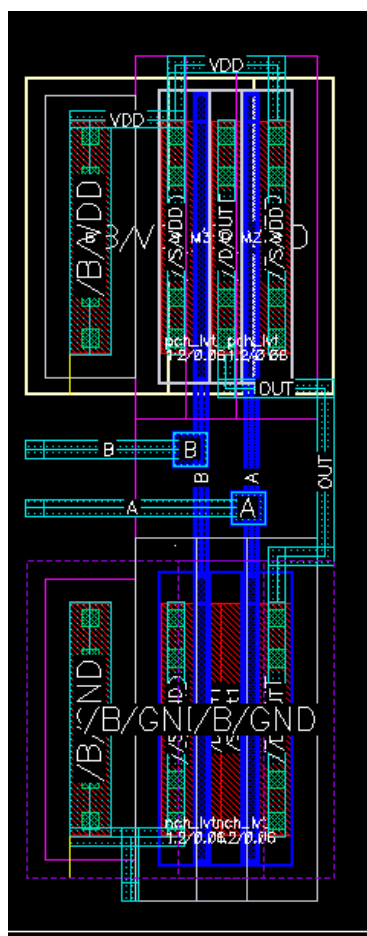


(a) NOT gate layout

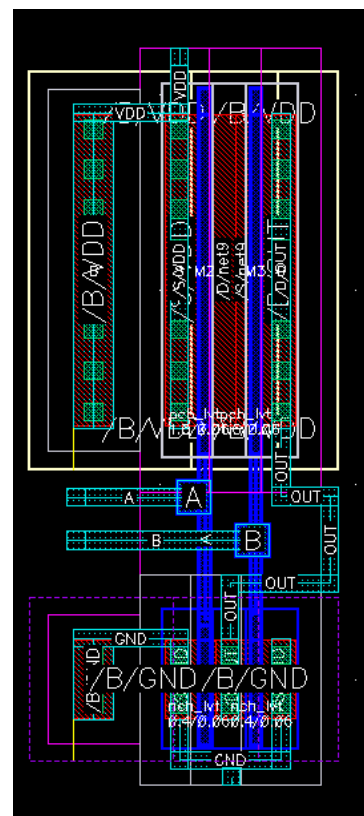


(b) XOR gate layout

Figure 4.1: Layouts of the NOT and XOR logic cells



(c) NAND gate layout



(d) NOR gate layout

Figure 4.2: Layouts of the NAND and NOR logic cells

4.1.1 Layout Dimensions and Area

The dimensions and corresponding layout areas of the basic CMOS logic cells are summarized in Table 4.1.

Table 4.1: Dimensions and area of basic CMOS logic cells.

Gate	Width (μm)	Height (μm)	Area (μm^2)
NOT	1.205	2.680	3.23
NAND	1.580	4.380	6.92
NOR	1.580	3.780	5.97
XOR	4.295	5.430	23.32

The optimized dimensions and areas of the standard cells are taken from the final layout implementation. The optimized layouts reduce the area compared with the earlier versions, with the largest reduction observed for the NOR gate.

4.1.2 Design Rule Check

The layouts of the basic CMOS cells were verified using Design Rule Check (DRC). The DRC results for the NOT, NAND, NOR, and XOR layouts were clean, confirming that the layouts

satisfy the applicable technology design rules.

Filter: Show All ☒ not_gate, 54 Results

Check / Cell	Results	Flat
Check G.4:PPI	2	
Check OD.DN.1:L	1	
Check DOD.R.1	1	
Check PO.DN.1:L	1	
Check PO.R.8	2	
Check DPO.R.1	1	
Check M9.DN.1:L	1	
Check CSR.R.1:NWi	1	
Check CSR.R.1:PPI	2	
Check CSR.R.1:NPI	2	
Check CSR.R.1:COi	17	
Check CSR.R.1:VTL_Ni	1	
Check CSR.R.1:VTL_Pi	1	
Check CSR.R.1:M1i	4	
Check CSR.R.1:M1_real	4	
Check CSR.R.1:ODi	2	
Check CSR.R.1:POi	1	
Check DM1.R.1	1	
Check DM2.R.1	1	
Check DM3.R.1	1	
Check DM4.R.1	1	
Check DM5.R.1	1	
Check DM6.R.1	1	
Check DM7.R.1	1	
Check DM8.R.1	1	
Check DM9.R.1	1	
Check ESD_WARN.1	1	
Check DENSITY_PRINT_FILES	0	0

(a) NOT gate DRC

Filter: Show All ☒ nand_gate, 87 Results (in 25 of 2)

Check / Cell	Results	Flat
Check DOD.R.1	1	1
Check PO.DN.1:L	1	1
Check PO.R.8	4	4
Check DPO.R.1	1	1
Check M9.DN.1:L	1	1
Check CSR.R.1:NWi	1	1
Check CSR.R.1:PPI	2	2
Check CSR.R.1:NPI	2	2
Check CSR.R.1:COi	43	44
Check CSR.R.1:VTL_Ni	1	1
Check CSR.R.1:VTL_Pi	1	1
Check CSR.R.1:M1i	6	7
Check CSR.R.1:M1_real	6	7
Check CSR.R.1:ODi	4	4
Check CSR.R.1:POi	3	4
Check DM1.R.1	1	1
Check DM2.R.1	1	1
Check DM3.R.1	1	1
Check DM4.R.1	1	1
Check DM5.R.1	1	1
Check DM6.R.1	1	1
Check DM7.R.1	1	1
Check DM8.R.1	1	1
Check DM9.R.1	1	1
Check ESD_WARN.1	1	1
Check DENSITY_PRINT_FILES	0	0

(b) NAND gate DRC

Filter: Show All ☒ nor_gate, 77 Results (in 25 of 28)

Check / Cell	Results	Flat
Check DOD.R.1	1	1
Check PO.DN.1:L	1	1
Check PO.R.8	4	4
Check DPO.R.1	1	1
Check M9.DN.1:L	1	1
Check CSR.R.1:NWi	1	1
Check CSR.R.1:PPI	2	2
Check CSR.R.1:NPI	2	2
Check CSR.R.1:COi	33	34
Check CSR.R.1:VTL_Ni	1	1
Check CSR.R.1:VTL_Pi	1	1
Check CSR.R.1:M1i	6	7
Check CSR.R.1:M1_real	6	7
Check CSR.R.1:ODi	4	4
Check CSR.R.1:POi	3	4
Check DM1.R.1	1	1
Check DM2.R.1	1	1
Check DM3.R.1	1	1
Check DM4.R.1	1	1
Check DM5.R.1	1	1
Check DM6.R.1	1	1
Check DM7.R.1	1	1
Check DM8.R.1	1	1
Check DM9.R.1	1	1
Check ESD_WARN.1	1	1
Check DENSITY_PRINT_FILES	0	0

(c) NOR gate DRC

Filter: Show All ☒ xor_gate, 203 Results (in 3)

Check / Cell	Results
Check OD.DN.1:L	1
Check DOD.R.1	1
Check PO.DN.1:L	1
Check PO.R.8	6
Check DPO.R.1	1
Check M9.DN.1:L	1
Check CSR.R.1:NWi	4
Check CSR.R.1:PPI	5
Check CSR.R.1:NPI	4
Check CSR.R.1:COi	73
Check CSR.R.1:VTL_Ni	2
Check CSR.R.1:VTL_Pi	4
Check CSR.R.1:M1i	24
Check CSR.R.1:M1_real	24
Check CSR.R.1:M2i	9
Check CSR.R.1:M2_real	9
Check CSR.R.1:VIA1i	3
Check CSR.R.1:ODi	8
Check CSR.R.1:POi	7
Check DM1.R.1	1
Check DM2.R.1	1
Check DM3.R.1	1

(d) XOR gate DRC

Figure 4.3: DRC verification of the basic CMOS logic cells.

4.1.3 Layout Versus Schematic Verification

LVS verification was performed for each basic CMOS cell to confirm that the extracted layout netlist matches the corresponding schematic.

Cell not_gate Summary (Clean)

CELL COMPARISON RESULTS (TOP LEVEL)



LAYOUT CELL NAME: not_gate
SOURCE CELL NAME: not_gate

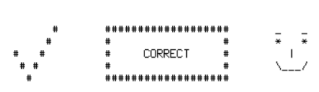
INITIAL NUMBERS OF OBJECTS

	Layout	Source	Component Type
Ports:	4	4	
Nets:	4	4	
Instances:	1	1	MN (4 pins)
	1	1	MP (4 pins)
Total Inst:	2	2	

(a) NOT gate LVS

Cell nand_gate Summary (Clean)

CELL COMPARISON RESULTS (TOP LEVEL)



LAYOUT CELL NAME: nand_gate
SOURCE CELL NAME: nand_gate

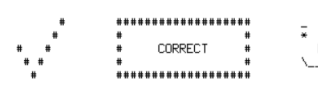
INITIAL NUMBERS OF OBJECTS

	Layout	Source	Component Type
Ports:	5	5	
Nets:	6	6	
Instances:	2	2	MN (4 pins)
	2	2	MP (4 pins)
Total Inst:	4	4	

(b) NAND gate LVS

Cell nor_gate Summary (Clean)

CELL COMPARISON RESULTS (TOP LEVEL)



LAYOUT CELL NAME: nor_gate
SOURCE CELL NAME: nor_gate

INITIAL NUMBERS OF OBJECTS

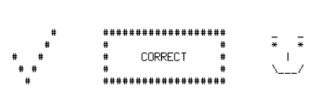
	Layout	Source	Component Type
Ports:	5	5	
Nets:	6	6	
Instances:	2	2	MN (4 pins)
	2	2	MP (4 pins)
Total Inst:	4	4	

NUMBERS OF OBJECTS AFTER TRANSFORMATION

(c) NOR gate LVS

Cell xor_gate Summary (Clean)

CELL COMPARISON RESULTS (TOP LEVEL)



LAYOUT CELL NAME: xor_gate
SOURCE CELL NAME: xor_gate

INITIAL NUMBERS OF OBJECTS

	Layout	Source	Component Type
Ports:	5	5	
Nets:	10	10	
Instances:	6	6	MN (4 pins)
	6	6	MP (4 pins)
Total Inst:	12	12	

(d) XOR gate LVS

Figure 4.4: LVS verification of the basic CMOS logic cells.

All four basic cells passed LVS verification.

4.2. Han–Carlson Adder Layout

The Han–Carlson adder was implemented using the designed Black Cell and Gray Cell layouts. The individual cells were optimized to reduce the overall area of the prefix network.

4.2.1 Black Cell Layout

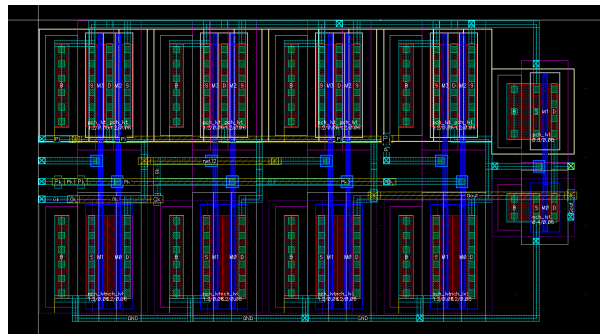


Figure 4.5: Black Cell layout.

4.2.2 Gray Cell Layout

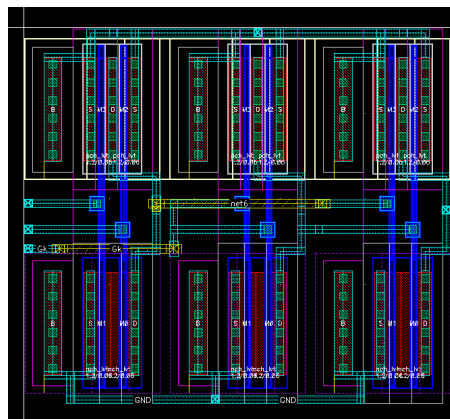


Figure 4.6: Gray Cell layout.

4.2.3 Han–Carlson Adder Layout

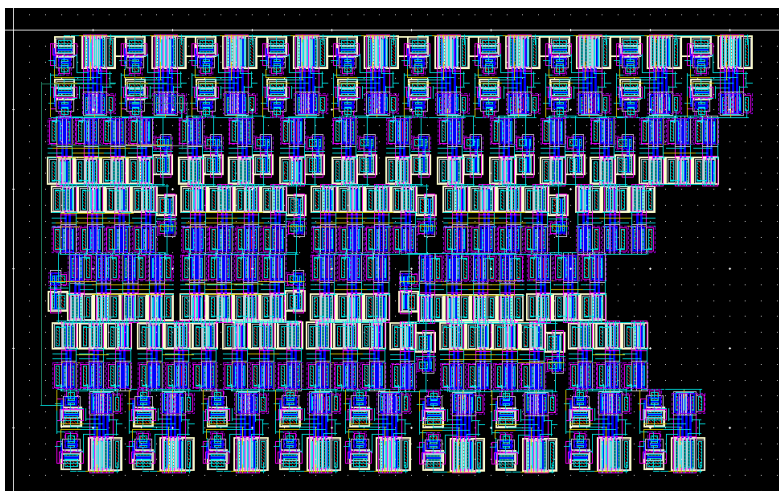


Figure 4.7: Han–Carlson adder layout.

4.2.4 Layout Dimensions and Area

The dimensions and layout areas of the Black Cell, Gray Cell, and complete Han–Carlson adder are summarized in Table 4.2.

Table 4.2: Dimensions and area of Han–Carlson adder blocks.

Block	Width (μm)	Height (μm)	Area (μm^2)
Black Cell	4.38	7.83	34.30
Gray Cell	4.38	4.98	21.81
Han–Carlson Adder	42.57	27.55	1172.8

The optimized Black Cell and Gray Cell layouts have dimensions of $4.38 \times 7.83 \mu\text{m}$ and $4.38 \times 4.98 \mu\text{m}$, respectively.

The complete Han–Carlson adder layout has a reported dimension of $27.55 \times 42.57 \mu\text{m}$ in the current implementation.

4.2.5 DRC Verification

DRC verification was performed on the Black Cell, Gray Cell, and Han–Carlson adder layouts.

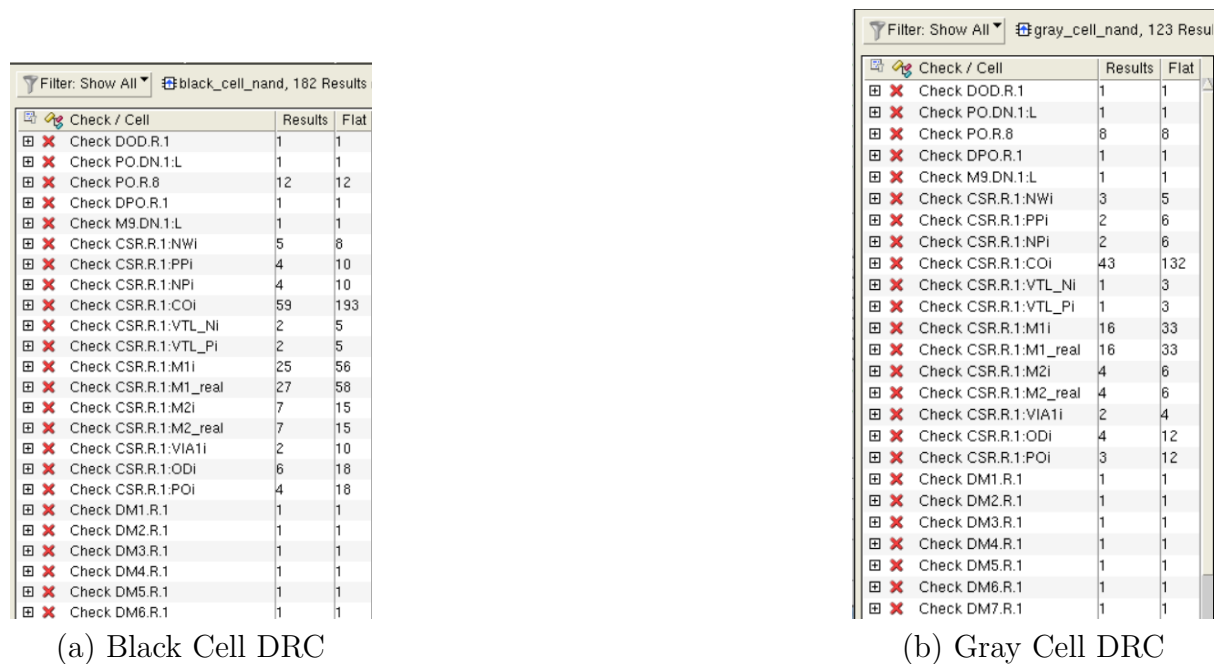


Figure 4.8: DRC verification of the Han–Carlson adder layouts.

4.2.6 LVS Verification

LVS verification was performed to confirm the correspondence between the Black Cell, Gray Cell, and Han–Carlson adder layouts and their respective schematics.

Cell black_cell_nand Summary (Clean)

CELL COMPARISON RESULTS (TOP LEVEL)

LAYOUT CELL NAME: black_cell_nand
SOURCE CELL NAME: black_cell_nand

INITIAL NUMBERS OF OBJECTS

	Layout	Source	Component Type
Ports:	8	8	
Nets:	15	15	
Instances:	9	9	MN (4 pins) MP (4 pins)
Total Inst:	18	18	

(a) Black Cell LVS

Cell gray_cell_nand Summary (Clean)

CELL COMPARISON RESULTS (TOP LEVEL)

LAYOUT CELL NAME: gray_cell_nand
SOURCE CELL NAME: gray_cell_nand

INITIAL NUMBERS OF OBJECTS

	Layout	Source	Component Type
Ports:	6	6	
Nets:	11	11	
Instances:	6	6	MN (4 pins) MP (4 pins)
Total Inst:	12	12	

(b) Gray Cell LVS

Figure 4.9: LVS verification of the Han–Carlson adder layouts.

The DRC and LVS checks were completed successfully for the designed adder blocks.

4.3. Multiplier Layout

The layout implementation of the multiplier datapath uses the designed Half Adder and Full Adder cells as constituent arithmetic blocks.

4.3.1 Half Adder Layout

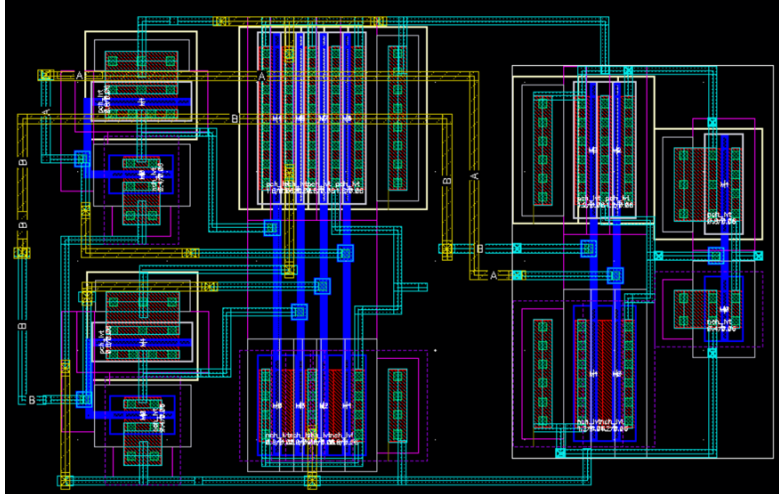


Figure 4.10: Half Adder layout.

4.3.2 Full Adder Layout

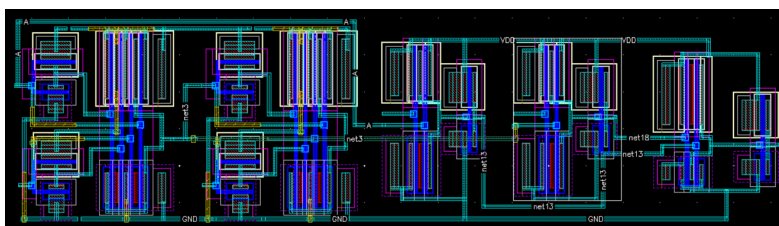


Figure 4.11: Full Adder layout.

4.3.3 Layout Dimensions and Area

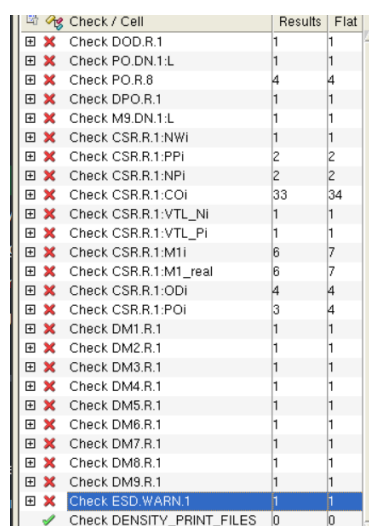
The dimensions and areas of the Half Adder and Full Adder layouts are summarized in Table 4.3. The final dimensions and areas of the Half Adder and Full Adder will be obtained from their completed layouts.

Table 4.3: Dimensions and area of multiplier arithmetic blocks.

Block	Width (μm)	Height (μm)	Area (μm^2)
Half Adder	8.45	5.13	43.34
Full Adder	20.15	5.13	103.36

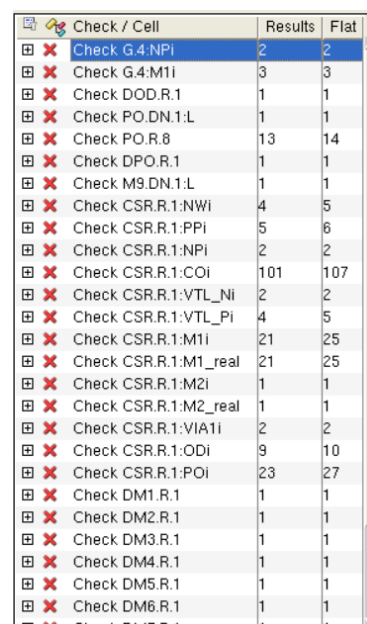
4.3.4 DRC Verification

DRC verification was performed for the Half Adder and Full Adder layouts. The DRC checks for the Half Adder and Full Adder layouts were clean.



Check / Cell	Results	Flat
Check DOD.R.1	1	1
Check PO.DN.1:L	1	1
Check PO.R.8	4	4
Check DPO.R.1	1	1
Check M9.DN.1:L	1	1
Check CSR.R.1:NWi	1	1
Check CSR.R.1:PPi	2	2
Check CSR.R.1:NPi	2	2
Check CSR.R.1:COi	33	34
Check CSR.R.1:VTL_Ni	1	1
Check CSR.R.1:VTL_Pi	1	1
Check CSR.R.1:M1i	6	7
Check CSR.R.1:M1_real	6	7
Check CSR.R.1:ODi	4	4
Check CSR.R.1:POi	3	4
Check DM1.R.1	1	1
Check DM2.R.1	1	1
Check DM3.R.1	1	1
Check DM4.R.1	1	1
Check DM5.R.1	1	1
Check DM6.R.1	1	1
Check DM7.R.1	1	1
Check DM8.R.1	1	1
Check DM9.R.1	1	1
Check ESD_WARN.1	1	1
Check DENSITY_PRINT_FILES	0	0

(a) Half Adder DRC



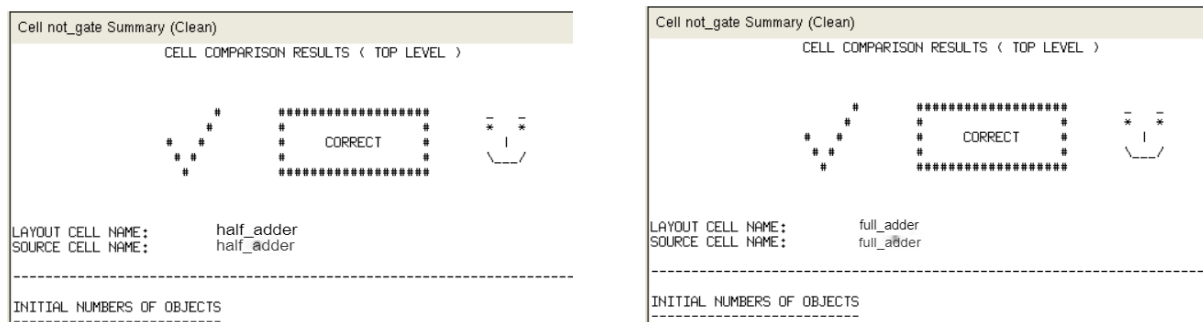
Check / Cell	Results	Flat
Check G.4:NPi	2	2
Check G.4:M1i	3	3
Check DOD.R.1	1	1
Check PO.DN.1:L	1	1
Check PO.R.8	13	14
Check DPO.R.1	1	1
Check M9.DN.1:L	1	1
Check CSR.R.1:NWi	4	5
Check CSR.R.1:PPi	5	6
Check CSR.R.1:NPi	2	2
Check CSR.R.1:COi	101	107
Check CSR.R.1:VTL_Ni	2	2
Check CSR.R.1:VTL_Pi	4	5
Check CSR.R.1:M1i	21	25
Check CSR.R.1:M1_real	21	25
Check CSR.R.1:M2i	1	1
Check CSR.R.1:M2_real	1	1
Check CSR.R.1:VIA1i	2	2
Check CSR.R.1:ODi	9	10
Check CSR.R.1:POi	23	27
Check DM1.R.1	1	1
Check DM2.R.1	1	1
Check DM3.R.1	1	1
Check DM4.R.1	1	1
Check DM5.R.1	1	1
Check DM6.R.1	1	1
Check DM7.R.1	1	1

(b) Full Adder DRC

Figure 4.12: DRC verification of the multiplier arithmetic blocks.

4.3.5 LVS Verification

LVS verification was performed for the Half Adder and Full Adder layouts. The LVS results confirm that the layouts correspond to their respective schematics.



(a) Half Adder LVS

(b) Full Adder LVS

Figure 4.13: LVS verification of the multiplier arithmetic blocks.

4.4. Summary of Layout Implementation

The layouts of the basic CMOS cells and arithmetic blocks were implemented and physically verified using DRC and LVS. The optimized standard-cell layouts achieved reduced area compared with the earlier implementations. The Black Cell and Gray Cell layouts were also optimized to reduce the area of the Han–Carlson prefix network.

The layout implementation forms the basis for the subsequent processing-element and top-level systolic-array physical implementation and post-layout verification.



CHAPTER

V

CONCLUSION

Chapter V



Conclusion

5.1. Timing Path Analysis

The timing target provided in this assignment was 100 MHz, that is, a clock period of 10 ns and the critical path achieved at the schematic level is 1.284 ns. Therefore, our design satisfies the timing requirement with a big margin. In the layout, the critical path is expected to be 15-20 % higher compared to the schematic. So, a critical path time of 1.541 ns would still satisfy the timing requirement. Therefore, our design can work at a frequency of 500 MHz as well with a sufficient margin.

5.2. Future Work

In the future, we would work on the following tasks:

- Complete layout of the Systolic Matrix Multiplication Engine
- Perform Parasitic Extraction (PEX) and analysis
- Perform post-layout simulation and testing
- Implement improvements in design and additional features as present in several research papers and patents

Referenced documents

- [1] Amazon Technologies, Inc., “Increasing Positive Clock Skew for Systolic Array Critical Path,”U.S. Patent US11347916B1, 2022. [Online]. Available: <https://patents.google.com/patent/US11347916B1/en>
- [2] Meta Platforms, Inc., “Data-Driven Column-Wise Clock Gating of Systolic Arrays,”U.S. Patent US11953966B1, 2024. [Online]. Available: <https://patents.google.com/patent/US11953966B1/en>
- [3] Alibaba, “Floorplan-Optimized Matrix Extension Architecture for Processors,”U.S. Patent Application US20240004830A1, 2024. [Online]. Available: <https://patents.google.com/patent/US20240004830A1/en>
- [4] ARM, “Technique for Distributing Routing into Superfluous Metal Section of an Integrated Circuit,”U.S. Patent US10452804B2, 2019. [Online]. Available: <https://patents.google.com/patent/US10452804B2/en>
- [5] Habana Labs Ltd., “Hardware Accelerator for Systolic Matrix Multiplication,”U.S. Patent US10915297B1, 2021. [Online]. Available: <https://patents.google.com/patent/US10915297B1/en>
- [6] Intel Corporation, “Systolic Array of Arbitrary Physical and Logical Depth,”U.S. Patent US12174783B2, 2024. [Online]. Available: <https://patents.google.com/patent/US12174783B2/en>
- [7] Samsung Electronics Co., Ltd., “Systolic Array and Accelerator Including the Same,”U.S. Patent Application US20220350861A1, 2022. [Online]. Available: <https://patents.google.com/patent/US20220350861A1/en>
- [8] Intel Corporation, “Systems and Methods for Reconfigurable Systolic Arrays,”U.S. Patent US11169957B2, 2020. [Online]. Available: <https://patents.google.com/patent/US11169957B2/en>
- [9] Intel Corporation, “Using Sparsity Metadata to Reduce Systolic Array Power Consumption,”U.S. Patent US12190158B2, 2021. [Online]. Available: <https://patents.google.com/patent/US12190158B2/en>

-
- [10] Intel Corporation, “Checksum-Based Fault Detection and Correction for a Matrix Compute Engine,” U.S. Patent Application US20240168841A1, 2022. [Online]. Available: <https://patents.google.com/patent/US20240168841A1/en>
- [11] Qualcomm Incorporated, “Clocking a Systolic Array on Both Edges of a Clock Signal,” U.S. Patent US12675433B2, 2026. [Online]. Available: <https://patents.google.com/patent/US12675433B2/en>
- [12] W.-K. Cheng, Z.-M. Yeh, H.-Y. Kao, and S.-H. Huang, “Cross-Mesh Clock Network Synthesis,” *Electronics*, vol. 12, no. 16, Art. no. 3410, 2023. [Online]. Available: <https://www.mdpi.com/2079-9292/12/16/3410>
- [13] S. Sun and C. Sechen, “Post-Layout Comparison of High Performance 64b Static Adders in Energy-Delay Space,” in *Proc. IEEE International Symposium on Low Power Electronics and Design (ISLPED)*, 2007.
- [14] B. Ramkumar and H. M. Kittur, “Faster and Energy-Efficient Signed Multipliers,” *VLSI Design*, vol. 2013, Article ID 495354, 2013, doi: 10.1155/2013/495354. [Online]. Available: <https://onlinelibrary.wiley.com/doi/10.1155/2013/495354>
- [15] “Design and Implementation of Compressor Based 32-bit Multipliers for MAC Architecture,” *International Journal of Innovative Technology and Exploring Engineering*, vol. 8, no. 9, 2019, doi: 10.35940/ijitee.I8517.078919. [Online]. Available: https://www.researchgate.net/publication/364660499_Design_and_Implementation_of_Compressor_based_32-bit_Multipliers_for_MAC_Architecture
- [16] “Comparison of Booth, Wallace Tree and Dadda Multipliers,” *IACSIT International Journal of Engineering and Technology*, vol. 4, no. 6, Dec. 2012. [Online]. Available: <https://ijetch.org/papers/482-C094.pdf>